Lab 1: Data Representation — Bit Manipulation

CSCI 3334 · UTRGV Fall 2026

Overview

Weight: 10% | **Out:** Wed Sep 9 | **Due:** Wed Sep 23

Implement 12 functions using only bitwise and integer operations. No loops, no conditionals, no macros. You may only use:

- Constants: `0x00`–`0xFF`
- Operators: `! ~ & ^ | + << >>`
- Variables and assignment
- `int` data type

Not allowed: `if`, `else`, `for`, `while`, `switch`, `?:`, function calls, macros, `long`, arrays, structs.

> **Important**
>
> This lab forces you to think in terms of **bits**, not values. Most functions can be solved in under 10 operations.

Setup

Get the starter code the same way you did for Lab 0 — if you already have the repo, `git pull` instead of cloning again:

```
git clone https://github.com/jpcoding/csci3334-labs.git
cd csci3334-labs/lab1
make                  # compile bits.c and check the rules
make check            # just the rules check
make check P=negate   # just one puzzle
```

All 12 puzzles are already stubbed in `bits.c`, and the file compiles before you write a line — every stub returns 2.

You'll edit `bits.c`. Each function has a comment specifying:

```
/*
 * bitXor - x^y using only ~ and &
 *   Example: bitXor(4, 5) = 1
 *   Legal ops: ~ &
 *   Max ops: 14
 *   Rating: 1
 */
int bitXor(int x, int y) {
    return 2;  // Replace this
}
```

Functions (10 required + 2 bonus)

The two marked *(bonus)* are 10 and 12; the other ten are required.

> **Note**

Start with Level 1 and 2

This goes out before two of the lectures it draws on. Levels 1 and 2 need only what you already have: `bitXor` through `isLessOrEqual` are all reachable on day one. `isTmax`, `isLessOrEqual` and `howManyBits` get easier after **Lecture 5** (Sep 14), and every `float_*` puzzle needs **Lecture 5b** (Sep 16) — a full week before the deadline. Do not save Level 1 for the end.

Level 1 — Warmup

#	Function	Description	Max Ops
1	`bitXor(x,y)`	`x ^ y` using only `~` and `&`	14
2	`tmin()`	Smallest two's complement integer	4
3	`isTmax(x)`	Returns 1 if `x` is the maximum two's complement integer	10
4	`allOddBits(x)`	Returns 1 if all odd-numbered bits are 1	12
5	`negate(x)`	Return `-x` without using `-`	5

Level 2 — Bit Counting

#	Function	Description	Max Ops
6	`isAsciiDigit(x)`	1 if `0x30 <= x <= 0x39`	15
7	`conditional(x,y,z)`	Same as `x ? y : z`	16
8	`isLessOrEqual(x,y)`	1 if `x <= y`	24

Level 3 — Float-Level

#	Function	Description	Max Ops
9	`float_neg(uf)`	Return `-f` as unsigned. If NaN, return `uf`.	10
10	`float_i2f(x)`	*(bonus)* Convert `int` to `float` bit pattern	30

Level 4 — Advanced

#	Function	Description	Max Ops
11	`howManyBits(x)`	Minimum bits needed to represent `x` in two's complement	90
12	`float_f2i(uf)`	*(bonus)* Convert `float` bit pattern to `int`	30

Hints

- `tmin()`: What does the bit pattern of `INT_MIN` look like?
- `bitXor(x,y)`: De Morgan's laws — `x ^ y = (x & ~y) | (~x & y)`. Now remove `|`.
- `negate(x)`: Two's complement negation: flip bits and add 1.
- For float puzzles: allowed ops include `if`, `while`, and `int`/`unsigned` casts, but **no** floating-point operations or unions.

Checking Your Work

`make check` runs `dlc.py`, which answers one question: **is your answer legal?** It reads each puzzle's rules out of the comment above it and reports the operators you used, your operator count against the budget, and anything banned — a loop, a conditional, a constant over `0xFF`.

```
OK    negate         2/5 ops
FAIL  isTmax  (line 43)
      uses 'while' — not allowed in this puzzle
      illegal operator(s): ==
      constant 0x7FFFFFFF exceeds 0xFF
```

It cannot tell you whether your answer is **correct**. That is graded after you submit, so the checking is yours to do:

- Every puzzle comment carries worked examples. Run them through your expression on paper.
- Then try the values that break things: `0`, `-1`, `TMin`, `TMax`, and for the float puzzles `0.0`, `-0.0`, infinity and NaN.
- Write a scratch `main()` in a separate file and print what your function returns. Do not leave it in `bits.c`.

An answer that is legal and wrong scores nothing, so spend your time on the second question, not the first.

Grading

Criterion	Points
Correctness — 10 required puzzles, graded after submission	90
Code style (comments, clarity)	10
Bonus: operator counts within limits (`make check`)	+10
Bonus: puzzles 10 and 12 (`float_i2f`, `float_f2i`)	+10

A correct solution that uses more operators than the stated limit still earns full correctness credit. The operator limit is a puzzle-golf challenge worth bonus points, not a penalty for solving the problem the straightforward way.

Submission

Upload `bits.c` to the Lab assignment on **Brightspace**. Submit the source file itself — not a screenshot, not a zip, not a link to a repo.

Your file must compile with the flags in the handout. Code that does not build scores the correctness points it earns when it does not build, which is none — check before you upload.

References

- CS:APP §2.1–2.4 (Integer representation, two's complement, bit-level operations)
- CS:APP §2.4 (Floating point)
- Bit Twiddling Hacks